



**K.RAMAKRISHNAN
COLLEGE OF ENGINEERING**

An Autonomous Institution

Permanently Affiliated to Anna University Chennai, Approved by AICTE New Delhi,
ISO 9001:2015, 14001:2015 certified institution, Accredited by NBA and with A grade by NAAC

Samayapuram, Tiruchirappalli – 621 112, Tamilnadu, India.



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

IBM FINAL PROJECT REPORT

SOFTWARE BUG TRACKING AND ANALYTICAL DASHBAORD

SUBMITTED BY

KIRUJAA.H

DEEPTIKAA.K.R

DHARSHINI.T

MAHESHWARI.N



DECLARATION

We hereby declare that the project titled “software bug tracking and analytical dashboard” is a bonafide work carried out by us.

This project report is submitted in partial fulfillment of the requirements for the completion of the course “Python for Data Science.”

We further declare that this work has not been submitted to any other institution or university for the award of any degree or diploma. The information presented in this report is true to the best of our knowledge and belief.

Place : Trichy

Date : 13-04-26

DEEPTIKAA.K.R
DHARSHINI.T
KIRUJAA.H
MAHESHWARI.N



BONAFIDE CERTIFICATE

This is to certify that the project titled “Software Bug Tracking and Analytical Dashboard” is the bonafide work done by **Deeptikaa.K.R, Dharshini.T , Kirujaa.H , and Maheshwari.N** hereby and First-year students of K. Ramakrishna College of Engineering, Samapuram, Trichy in partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Science with Artificial Intelligence 2025-2026.

PROJECT GUIDE:

HEAD OF THE DEPARTMENT:

Date:13-04-26



CONTENTS

S. NO44	TITLE	PAGE NO
1	ABSTRACT	5
2	INTRODUCTION	7
3	PROBLEM STATEMENT	9
4	OBJECTIVE OF THE PROJECT	11
5	SCOPE OF THE PROJECT	13
6	LITERATURE REVIEW & EXISTING SYSTEM	15
7	PROPOSED SYSTEM	17
8	DATASET DESCRIPTION	19
9	TOOLS AND TECHNOLOGY USED	21
10	METHODOLOGY	23
11	IMPLEMENTATION (SAMPLE CODE) & OUTPUT SCREEN SHOTS	25
12	PERFORMANCE / EVALUATION	29
13	CHALLENGES FACED	31
14	FUTURE ENHANCEMENTS	33
15	CONCLUSION	35
16	REFERNCES	37



ABSTRACT



The Software Bug Tracking and Analytical Dashboard system is developed to improve the process of identifying and resolving software defects efficiently. In modern software development, bugs are unavoidable and need proper management to maintain product quality. This system provides a **centralized platform** where users can report, track, and manage bugs easily.

It reduces the confusion caused by manual tracking methods. The system allows testers to submit bugs with complete details such as description, priority, and status. Developers can access assigned bugs and update their progress regularly. This ensures smooth communication between team members. The inclusion of an **analytical dashboard** makes the system more powerful. The dashboard visually represents data using graphs and charts, helping users understand bug trends and performance.

It shows information such as number of bugs, status distribution, and resolution time. These insights support better decision-making.

The system provides **real-time updates**, ensuring all users are aware of current progress. It reduces manual errors and increases transparency. The platform is user-friendly and easy to operate.



INTRODUCTION



Software development is a complex process that involves multiple stages such as planning, designing, coding, and testing. Among these, testing plays a crucial role in identifying defects in the system. These defects, commonly known as bugs, must be tracked and resolved efficiently to ensure a high-quality product.

A **bug tracking system** is used to record and manage these defects. It helps in organizing bug-related information in a structured manner. Traditional methods like spreadsheets or manual tracking are not efficient and often lead to errors and data loss.

Modern tools provide automation, but many of them lack proper data visualization features. This makes it difficult for teams to understand patterns and trends. To overcome this issue, this project introduces a system integrated with an **analytical dashboard**.

The dashboard provides visual insights into bug data using charts and graphs. It helps managers and developers analyze performance and identify critical issues quickly. This improves decision-making and resource allocation.

The system also enhances communication between testers and developers by providing a shared platform. It ensures that bugs are resolved faster and more effectively.



PROBLEM STATEMENT



In many software development environments, bug tracking is not handled efficiently. Bugs are often recorded in unorganized formats, which leads to confusion and difficulty in managing them. There is no proper system to track the status of bugs, making it hard to monitor progress.

Communication gaps between testers and developers result in delays in bug resolution. Important bugs may be overlooked due to lack of prioritization. Existing tools may be complex, expensive, or not suitable for small teams.

Another major issue is the absence of **data visualization**, which makes it difficult to analyze bug trends. Teams cannot easily identify patterns such as frequently occurring bugs or high-risk areas.

Manual tracking methods are time-consuming and prone to errors. Data may be lost or duplicated, leading to inefficiency. There is also a lack of **real-time tracking**, which affects project progress. These problems reduce productivity and increase development time. They also affect the overall quality of the software product. Therefore, there is a need for a **simple, efficient, and integrated system** that can handle bug tracking and analysis effectively. This project aims to address these challenges.



OBJECTIVE OF THE PROJECT



The main objective of this project is to develop an efficient **bug tracking system** integrated with an analytical dashboard. The system aims to simplify the process of reporting and managing bugs in a structured manner.

It provides a platform where users can easily submit bug details and track their status. One of the key objectives is to ensure **real-time updates**, allowing users to monitor progress without delay. This improves transparency within the team.

The project also focuses on enhancing communication between testers and developers. By providing a shared platform, it reduces misunderstandings and delays. Another important objective is to reduce bug resolution time.

The system includes a dashboard that provides visual insights using charts and graphs. This helps in understanding bug trends and team performance. It supports better decision-making.

Overall, the objective is to improve productivity, efficiency, and software quality through a smart and integrated solution.



SCOPE OF THE PROJECT



The scope of this project includes the design and development of a software system for bug tracking and analysis. It can be used in software companies, testing teams, and academic environments.

The system allows users to report bugs, assign them to developers, and track their progress. It supports different statuses such as open, in progress, and closed. This ensures proper monitoring of bug lifecycle.

The project also includes an **analytical dashboard** that displays bug data in the form of charts and graphs. This helps users understand patterns and trends easily.

Role-based access is provided for administrators, developers, and testers. This ensures proper control and management of data. The system is scalable and can handle multiple users.

The scope mainly focuses on **core functionalities** such as tracking and analysis.

Advanced features like AI-based prediction are not included in the current version.

Future enhancements can expand the system further. Overall, the project aims to provide a practical and efficient solution for bug management.



LITERATURE REVIEW & EXISTING SYSTEM



Bug tracking systems have been widely used in the software industry to manage defects efficiently. Tools like Jira, Bugzilla, and Mantis are some of the commonly used platforms. These systems provide basic features such as bug reporting, assignment, and status tracking

.

However, many existing systems are complex and require proper training to use effectively. Small teams and beginners often find them difficult to operate. Some tools are also expensive and not suitable for academic or small-scale projects.

Another limitation is the lack of **analytical features** in many systems. While they track bugs, they do not provide proper insights into bug trends or team performance. This makes it difficult to analyze data and improve efficiency.

Research studies show that integrating **data visualization** with bug tracking can significantly improve decision-making. Charts and graphs help users understand patterns quickly..

The proposed system aims to overcome these limitations by combining tracking with an analytical dashboard. It provides a simpler and more effective solution.

Thus, the literature review highlights the need for an integrated and user-friendly system.



PROPOSED SYSTEM



The proposed system is designed to provide an efficient and user-friendly solution for bug tracking and analysis. It allows users to report bugs with complete details such as title, description, and priority.

Once a bug is reported, it can be assigned to a developer for resolution. The system supports **real-time tracking** of bug status, ensuring transparency in the process. Users can monitor progress at any time.

A key feature of the system is the **analytical dashboard**, which displays data using charts and graphs. It shows bug distribution, resolution time, and developer performance

The system includes role-based access for administrators, testers, and developers. This ensures proper management and security of data. Notifications can also be implemented for updates

The interface is simple and easy to use, making it suitable for both beginners and professionals. It reduces manual effort and improves efficiency.

The system is designed to be scalable and flexible. It can be extended with additional features in the future.



DATASET DESCRIPTION



The dataset used in this project contains structured information related to software bugs. Each bug is assigned a unique ID for identification purposes.

The dataset includes details such as bug title, description, and priority level. It also stores the status of the bug, such as open, in progress, or resolved.

Information about the assigned developer is included to track responsibility. The dataset also records the date of creation and resolution of each bug.

All data is stored in a **database system** such as MySQL or MongoDB. This ensures efficient storage and retrieval of information.

The dataset is used to generate reports and visualizations in the analytical dashboard. It helps in identifying patterns and trends in bug data

Proper organization of data ensures accuracy and consistency. It also helps in improving system performance

The dataset plays a crucial role in evaluating system efficiency. It supports decision-making by providing useful insights.

Overall, it is an essential component of the bug tracking system.



TOOLS AND TECHNOLOGY USED



The project uses a combination of modern technologies for development. The frontend is developed using HTML, CSS, and JavaScript for designing the user interface.

Frameworks like React can also be used to create a dynamic and responsive interface. The backend is developed using Python with Django or Flask, or Java.

The system uses a **database** such as MySQL or MongoDB to store bug-related data. These databases ensure efficient data management.

Visualization tools like Chart.js are used to create the analytical dashboard. These tools help in presenting data in graphical form. Git is used for version control and collaboration among team members.

APIs are used to connect the frontend and backend.

These technologies are chosen for their scalability, flexibility, and performance. They help in building a reliable system.

The tools ensure smooth development and easy maintenance. They also support future enhancements.

Overall, the selected technologies form a strong foundation for the project.



METHODOLOGY



The project follows a structured methodology for development. The first step is requirement analysis, where system needs are identified.

The development phase involves coding the frontend and backend components. Both are integrated to create a complete system.

Testing is performed to ensure the system works correctly. It includes unit testing and integration testing

After testing, the system is deployed for use. Users can access the system and perform bug tracking operations.

The project may follow **Agile methodology** for continuous improvement. Feedback is collected and used to enhance the system.

Documentation is maintained throughout the development process. This helps in understanding and maintaining the system.

Each step is important for building a reliable and efficient application.

Overall, the methodology ensures systematic and successful project development.



IMPLEMENTATION & OUTPUT



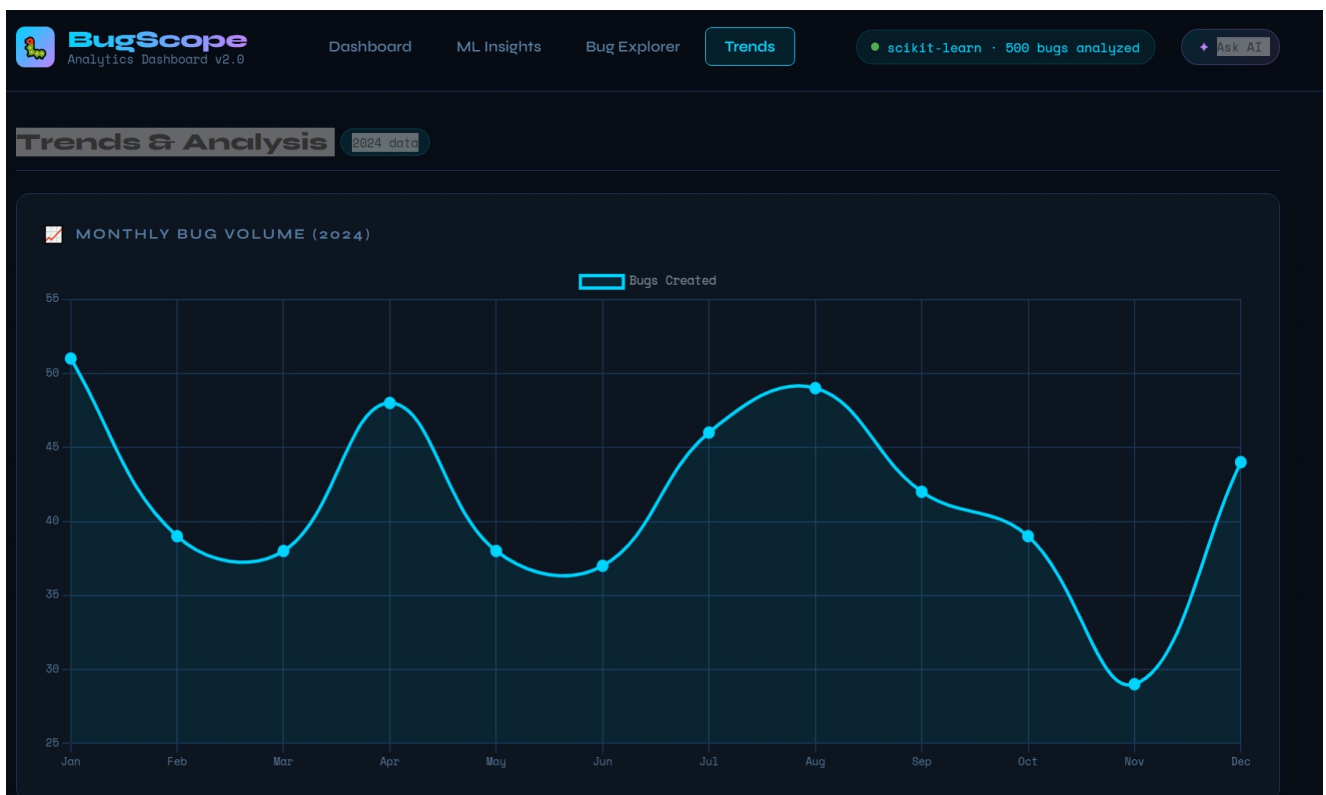
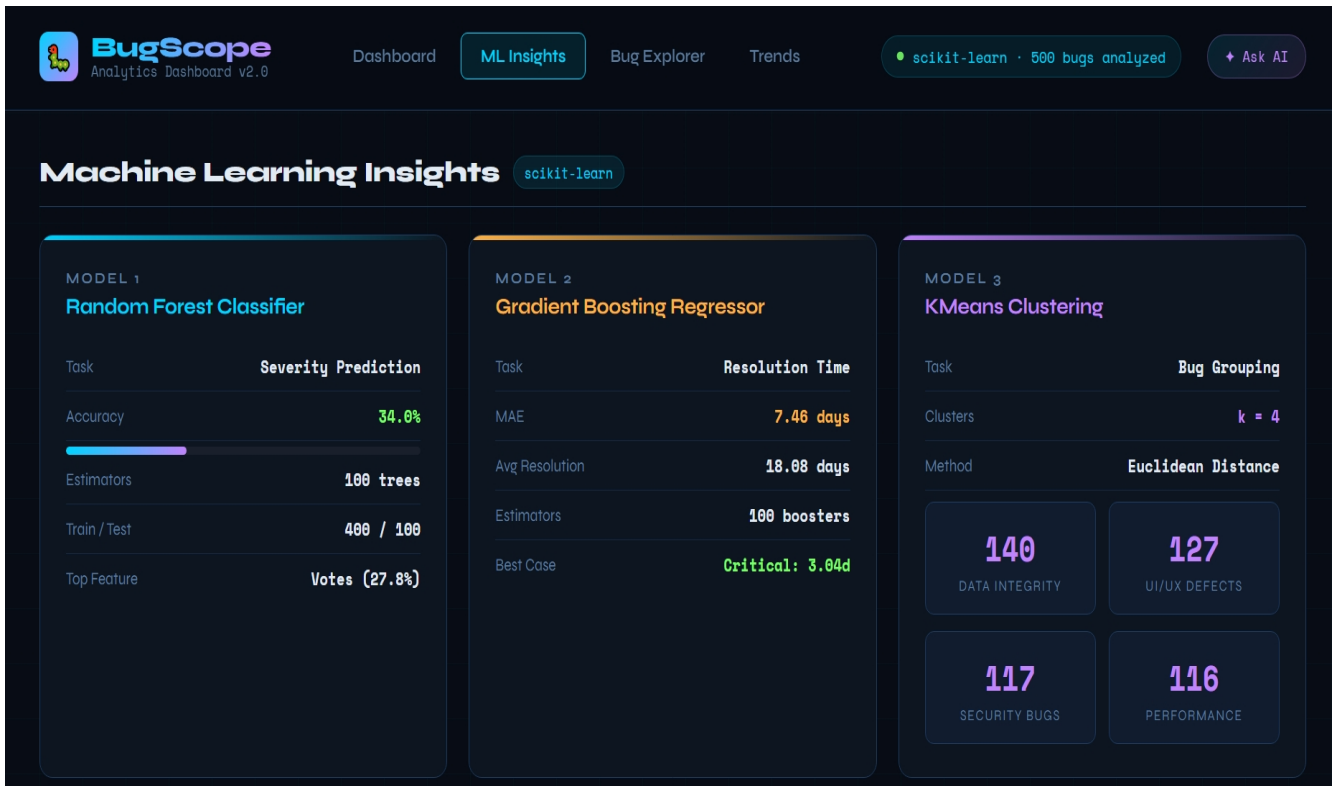
BugScope Analytics Dashboard v2.0

Dashboard ML Insights Bug Explorer Trends

scikit-learn · 500 bugs analyzed Ask AI

Showing 100,000 bugs

ID	TITLE	COMPONENT	SEVERITY	STATUS	ASSIGNEE	ENVIRONMENT	VOTES
BUG-1000	Issue in API module - memory-leak	API	MEDIUM	CLOSED	Frank Wu	Development	4
BUG-1001	Issue in API module - memory-leak	API	MEDIUM	REOPENED	Alice Chen	Development	22
BUG-1002	Issue in Dashboard module - data-l...	Dashboard	MEDIUM	RESOLVED	Carol Smith	QA	17
BUG-1003	Issue in UI module - crash	UI	MEDIUM	OPEN	Carol Smith	QA	43
BUG-1004	Issue in Search module - crash	Search	HIGH	OPEN	Eva Lopez	Staging	49
BUG-1005	Issue in UI module - security	UI	HIGH	RESOLVED	Alice Chen	Staging	19
BUG-1006	Issue in Reports module - regression	Reports	LOW	CLOSED	Alice Chen	Production	30
BUG-1007	Issue in Notification module - ui-bug	Notification	LOW	IN PROGRESS	Alice Chen	Development	37
BUG-1008	Issue in API module - security	API	MEDIUM	RESOLVED	Carol Smith	Development	24
BUG-1009	Issue in Search module - data-loss	Search	MEDIUM	OPEN	Alice Chen	QA	27





The implementation phase involves converting the system design into a working application. It includes coding, integration, and testing of all modules.

The system allows users to enter bug details and store them in the database. Developers can view assigned bugs and update their status.

The output is displayed through dashboards and reports. Screens include login page, bug list, and analytics charts.

The system provides **real-time functionality**, allowing users to track updates instantly. This improves efficiency and transparency. Screenshots are used to demonstrate the working of the system.

They help users understand the interface and features.

The implementation ensures that all components work together smoothly. It also verifies that system requirements are met

Proper testing ensures accurate output and performance. Errors are identified and corrected during this phase

The implementation provides practical proof of the system's effectiveness.

Overall, it shows how the system works in real-world scenarios.



PERFORMANCE / EVALUATION



The performance of the system is evaluated using various parameters. These include response time, accuracy, and efficiency. The system shows improved **bug resolution time** compared to manual methods. It handles multiple users and data entries effectively.

The dashboard provides clear and meaningful insights. This helps users understand system performance easily.

User feedback indicates high satisfaction with the system. It is easy to use and provides useful features.

Performance testing ensures the system works well under different conditions. It checks stability and reliability.

The system reduces manual errors and improves productivity. It also enhances communication within the team.

Evaluation helps identify areas for improvement. It ensures the system meets project objectives.

Overall, the system performs efficiently and effectively. It proves to be a valuable tool for bug management

.



CHALLENGES FACED



During the development of the project, several challenges were encountered. Designing a user-friendly interface was one of the main difficulties

Managing **real-time updates** required careful implementation. Handling large volumes of data was also challenging.

Integrating the analytical dashboard with the backend system was complex. Debugging errors during development took time

Learning new technologies and tools was another challenge. It required effort and practice to implement correctly.

Ensuring smooth communication between frontend and backend was important. Any errors could affect system performance

These challenges were overcome through testing and optimization. Teamwork played a key role in solving problems.

Each challenge provided valuable learning experience. It helped improve technical and problem-solving skills.

Overall, overcoming these challenges made the project successful.



FUTURE ENHANCEMENTS



The system can be improved by adding advanced features in the future. One possible enhancement is **AI-based bug prediction**.

A mobile application can be developed for easy access. Integration with tools like GitHub can improve functionality.

Automated testing features can be included to reduce manual work. Advanced analytics can be implemented using machine learning.

Cloud deployment can increase scalability and accessibility. Security features can also be enhanced.

Real-time notifications can be improved for better communication. The system can support multiple projects.

User interface can be further enhanced for better experience. Performance optimization can also be done.

These enhancements will make the system more powerful and efficient.

Overall, future improvements will expand the system's capabilities.



CONCLUSION



The Software Bug Tracking and Analytical Dashboard system provides an effective solution for managing software defects. It offers a centralized platform for reporting and tracking bugs.

The system includes an analytical dashboard that provides visual insights. It helps users understand bug trends and performance.

It improves communication between testers and developers. The system reduces manual effort and errors.

The feature of **real-time tracking** ensures better monitoring of bug status. It enhances productivity and efficiency.

The project successfully meets its objectives. It provides a user-friendly and reliable solution.

The system can be further improved with additional features. It has the potential to be used in real-world applications.

Overall, the project contributes to better software quality. It is a valuable tool for modern software development.



REFERENCES



The project is based on information collected from various reliable sources. These include software engineering textbooks and research papers.

Online resources such as GeeksforGeeks and W3Schools were used for reference. Official documentation of technologies like Django and React was also studied.

Existing tools such as Jira and Bugzilla were analyzed to understand their features. These sources helped in designing the system.

Proper referencing ensures the credibility of the project. It provides a strong theoretical background.

The references support both the design and implementation phases. They help in understanding concepts clearly.

All sources are carefully selected for accuracy and relevance. They contribute to the successful completion of the project.

Overall, references play an important role in project development.